

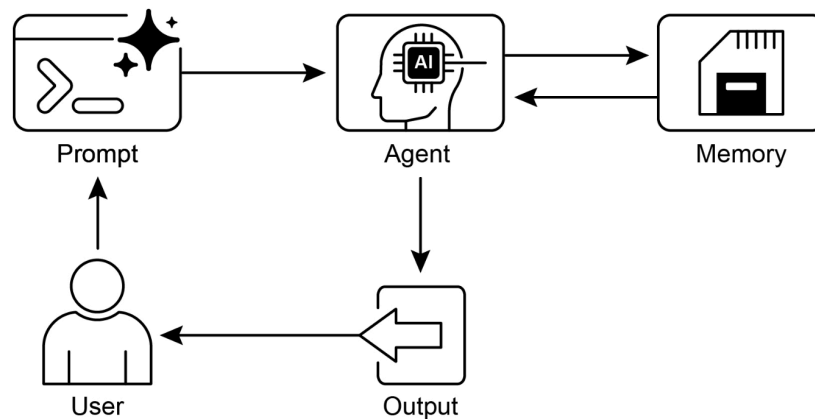


Fig.1: Memory management design pattern

Key Takeaways

To quickly recap the main points about memory management:

- Memory is super important for agents to keep track of things, learn, and personalize interactions.
- Conversational AI relies on both short-term memory for immediate context within a single chat and long-term memory for persistent knowledge across multiple sessions.
- Short-term memory (the immediate stuff) is temporary, often limited by the LLM's context window or how the framework passes context.
- Long-term memory (the stuff that sticks around) saves info across different chats using outside storage like vector databases and is accessed by searching.

- Frameworks like ADK have specific parts like Session (the chat thread), State (temporary chat data), and MemoryService (the searchable long-term knowledge) to manage memory.
- ADK's SessionService handles the whole life of a chat session, including its history (events) and temporary data (state).
- ADK's session.state is a dictionary for temporary chat data. Prefixes (user:, app:, temp:) tell you where the data belongs and if it sticks around.
- In ADK, you should update state by using EventActions.state_delta or output_key when adding events, not by changing the state dictionary directly.
- ADK's MemoryService is for putting info into long-term storage and letting agents search it, often using tools.
- LangChain offers practical tools like ConversationBufferMemory to automatically inject the history of a single conversation into a prompt, enabling an agent to recall immediate context.
- LangGraph enables advanced, long-term memory by using a store to save and retrieve semantic facts, episodic experiences, or even updatable procedural rules across different user sessions.
- Memory Bank is a managed service that provides agents with persistent, long-term memory by automatically extracting, storing, and recalling user-specific information to enable personalized, continuous conversations across frameworks like Google's ADK, LangGraph, and CrewAI.

Conclusion

This chapter dove into the really important job of memory management for agent systems, showing the difference between the short-lived context and the knowledge that sticks around for a long time. We talked about how these types of memory are set up and where you see them used in building smarter agents that can remember things. We took a detailed look at how Google ADK gives you specific pieces like Session, State, and MemoryService to handle this. Now that we've covered how agents can remember things, both short-term and long-term, we can move on to how they can learn and adapt. The next pattern "Learning and Adaptation" is about an agent changing how it thinks, acts, or what it knows, all based on new experiences or data.

References

1. ADK Memory, <https://google.github.io/adk-docs/sessions/memory/>

2. LangGraph Memory,
<https://langchain-ai.github.io/langgraph/concepts/memory/>
3. Vertex AI Agent Engine Memory Bank,
<https://cloud.google.com/blog/products/ai-machine-learning/vertex-ai-memory-bank-in-public-preview>

Chapter 9: Learning and Adaptation

Learning and adaptation are pivotal for enhancing the capabilities of artificial intelligence agents. These processes enable agents to evolve beyond predefined parameters, allowing them to improve autonomously through experience and environmental interaction. By learning and adapting, agents can effectively manage novel situations and optimize their performance without constant manual intervention. This chapter explores the principles and mechanisms underpinning agent learning and adaptation in detail.

The big picture

Agents learn and adapt by changing their thinking, actions, or knowledge based on new experiences and data. This allows agents to evolve from simply following instructions to becoming smarter over time.

- **Reinforcement Learning:** Agents try actions and receive rewards for positive outcomes and penalties for negative ones, learning optimal behaviors in changing situations. Useful for agents controlling robots or playing games.
- **Supervised Learning:** Agents learn from labeled examples, connecting inputs to desired outputs, enabling tasks like decision-making and pattern recognition. Ideal for agents sorting emails or predicting trends.
- **Unsupervised Learning:** Agents discover hidden connections and patterns in unlabeled data, aiding in insights, organization, and creating a mental map of their environment. Useful for agents exploring data without specific guidance.
- **Few-Shot/Zero-Shot Learning with LLM-Based Agents:** Agents leveraging LLMs can quickly adapt to new tasks with minimal examples or clear instructions, enabling rapid responses to new commands or situations.
- **Online Learning:** Agents continuously update knowledge with new data, essential for real-time reactions and ongoing adaptation in dynamic environments. Critical for agents processing continuous data streams.
- **Memory-Based Learning:** Agents recall past experiences to adjust current actions in similar situations, enhancing context awareness and decision-making. Effective for agents with memory recall capabilities.

Agents adapt by changing strategy, understanding, or goals based on learning. This is vital for agents in unpredictable, changing, or new environments.

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm used to train agents in environments with a continuous range of actions, like controlling a robot's joints or a character in a game. Its main goal is to reliably and stably improve an agent's decision-making strategy, known as its policy.

The core idea behind PPO is to make small, careful updates to the agent's policy. It avoids drastic changes that could cause performance to collapse. Here's how it works:

1. **Collect Data:** The agent interacts with its environment (e.g., plays a game) using its current policy and collects a batch of experiences (state, action, reward).
2. **Evaluate a "Surrogate" Goal:** PPO calculates how a potential policy update would change the expected reward. However, instead of just maximizing this reward, it uses a special "clipped" objective function.
3. **The "Clipping" Mechanism:** This is the key to PPO's stability. It creates a "trust region" or a safe zone around the current policy. The algorithm is prevented from making an update that is too different from the current strategy. This clipping acts like a safety brake, ensuring the agent doesn't take a huge, risky step that undoes its learning.

In short, PPO balances improving performance with staying close to a known, working strategy, which prevents catastrophic failures during training and leads to more stable learning.

Direct Preference Optimization (DPO) is a more recent method designed specifically for aligning Large Language Models (LLMs) with human preferences. It offers a simpler, more direct alternative to using PPO for this task.

To understand DPO, it helps to first understand the traditional PPO-based alignment method:

- **The PPO Approach (Two-Step Process):**
 1. **Train a Reward Model:** First, you collect human feedback data where people rate or compare different LLM responses (e.g., "Response A is better than Response B"). This data is used to train a separate AI model, called a reward model, whose job is to predict what score a human would give to any new response.
 2. **Fine-Tune with PPO:** Next, the LLM is fine-tuned using PPO. The LLM's goal is to generate responses that get the highest possible score from

the reward model. The reward model acts as the "judge" in the training game.

This two-step process can be complex and unstable. For instance, the LLM might find a loophole and learn to "hack" the reward model to get high scores for bad responses.

- The DPO Approach (Direct Process): DPO skips the reward model entirely. Instead of translating human preferences into a reward score and then optimizing for that score, DPO uses the preference data directly to update the LLM's policy.
- It works by using a mathematical relationship that directly links preference data to the optimal policy. It essentially teaches the model: "Increase the probability of generating responses like the *preferred* one and decrease the probability of generating ones like the *disfavored* one."

In essence, DPO simplifies alignment by directly optimizing the language model on human preference data. This avoids the complexity and potential instability of training and using a separate reward model, making the alignment process more efficient and robust.

Practical Applications & Use Cases

Adaptive agents exhibit enhanced performance in variable environments through iterative updates driven by experiential data.

- **Personalized assistant agents** refine interaction protocols through longitudinal analysis of individual user behaviors, ensuring highly optimized response generation.
- **Trading bot agents** optimize decision-making algorithms by dynamically adjusting model parameters based on high-resolution, real-time market data, thereby maximizing financial returns and mitigating risk factors.
- **Application agents** optimize user interface and functionality through dynamic modification based on observed user behavior, resulting in increased user engagement and system intuitiveness.
- **Robotic and autonomous vehicle agents** enhance navigation and response capabilities by integrating sensor data and historical action analysis, enabling safe and efficient operation across diverse environmental conditions.
- **Fraud detection agents** improve anomaly detection by refining predictive models with newly identified fraudulent patterns, enhancing system security

and minimizing financial losses.

- **Recommendation agents** improve content selection precision by employing user preference learning algorithms, providing highly individualized and contextually relevant recommendations.
- **Game AI agents** enhance player engagement by dynamically adapting strategic algorithms, thereby increasing game complexity and challenge.
- **Knowledge Base Learning Agents:** Agents can leverage Retrieval Augmented Generation (RAG) to maintain a dynamic knowledge base of problem descriptions and proven solutions (see the Chapter 14). By storing successful strategies and challenges encountered, the agent can reference this data during decision-making, enabling it to adapt to new situations more effectively by applying previously successful patterns or avoiding known pitfalls.

Case Study: The Self-Improving Coding Agent (SICA)

The Self-Improving Coding Agent (SICA), developed by Maxime Robeyns, Laurence Aitchison, and Martin Szummer, represents an advancement in agent-based learning, demonstrating the capacity for an agent to modify its own source code. This contrasts with traditional approaches where one agent might train another; SICA acts as both the modifier and the modified entity, iteratively refining its code base to improve performance across various coding challenges.

SICA's self-improvement operates through an iterative cycle (see Fig.1). Initially, SICA reviews an archive of its past versions and their performance on benchmark tests. It selects the version with the highest performance score, calculated based on a weighted formula considering success, time, and computational cost. This selected version then undertakes the next round of self-modification. It analyzes the archive to identify potential improvements and then directly alters its codebase. The modified agent is subsequently tested against benchmarks, with the results recorded in the archive. This process repeats, facilitating learning directly from past performance. This self-improvement mechanism allows SICA to evolve its capabilities without requiring traditional training paradigms.

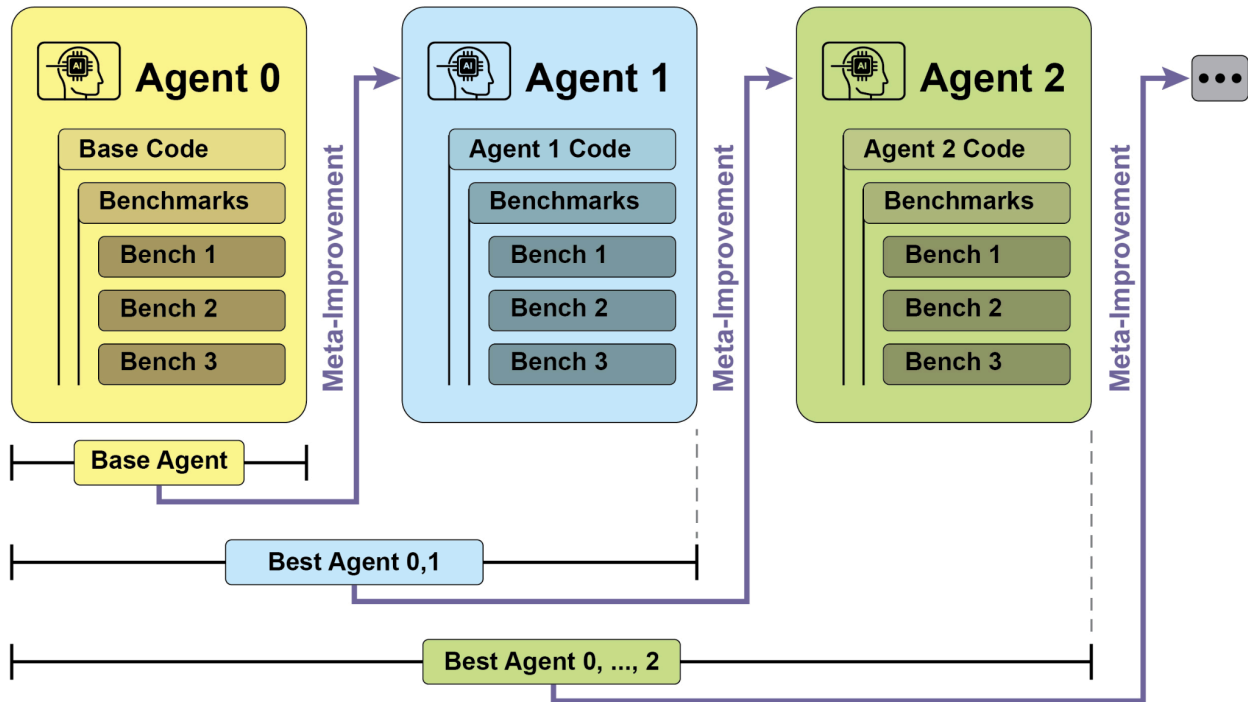


Fig.1: SICA's self-improvement, learning and adapting based on its past versions

SICA underwent significant self-improvement, leading to advancements in code editing and navigation. Initially, SICA utilized a basic file-overwriting approach for code changes. It subsequently developed a "Smart Editor" capable of more intelligent and contextual edits. This evolved into a "Diff-Enhanced Smart Editor," incorporating diffs for targeted modifications and pattern-based editing, and a "Quick Overwrite Tool" to reduce processing demands.

SICA further implemented "Minimal Diff Output Optimization" and "Context-Sensitive Diff Minimization," using Abstract Syntax Tree (AST) parsing for efficiency. Additionally, a "SmartEditor Input Normalizer" was added. In terms of navigation, SICA independently created an "AST Symbol Locator," using the code's structural map (AST) to identify definitions within the codebase. Later, a "Hybrid Symbol Locator" was developed, combining a quick search with AST checking. This was further optimized via "Optimized AST Parsing in Hybrid Symbol Locator" to focus on relevant code sections, improving search speed.(see Fig. 2)

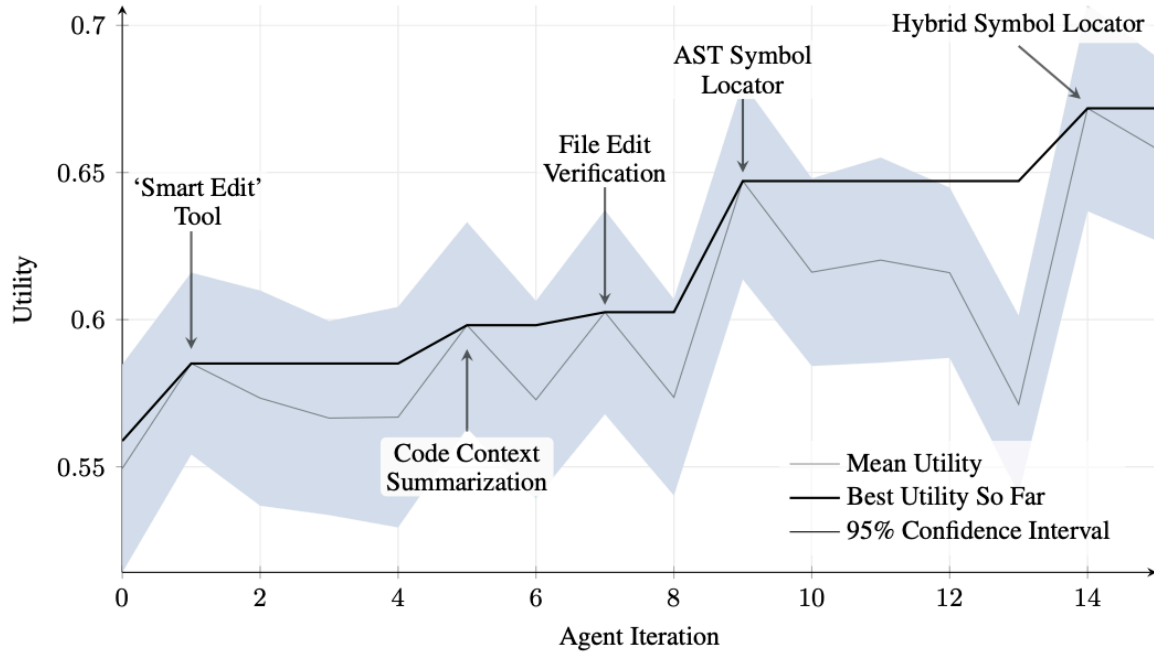


Fig.2 : Performance across iterations. Key improvements are annotated with their corresponding tool or agent modifications. (courtesy of Maxime Robeyns , Martin Szummer , Laurence Aitchison)

SICA's architecture comprises a foundational toolkit for basic file operations, command execution, and arithmetic calculations. It includes mechanisms for result submission and the invocation of specialized sub-agents (coding, problem-solving, and reasoning). These sub-agents decompose complex tasks and manage the LLM's context length, especially during extended improvement cycles.

An asynchronous overseer, another LLM, monitors SICA's behavior, identifying potential issues such as loops or stagnation. It communicates with SICA and can intervene to halt execution if necessary. The overseer receives a detailed report of SICA's actions, including a callgraph and a log of messages and tool actions, to identify patterns and inefficiencies.

SICA's LLM organizes information within its context window, its short-term memory, in a structured manner crucial to its operation. This structure includes a System Prompt defining agent goals, tool and sub-agent documentation, and system instructions. A Core Prompt contains the problem statement or instruction, content of open files, and a directory map. Assistant Messages record the agent's step-by-step reasoning, tool and sub-agent call records and results, and overseer communications. This organization facilitates efficient information flow, enhancing LLM operation and

reducing processing time and costs. Initially, file changes were recorded as diffs, showing only modifications and periodically consolidated.

SICA: A Look at the Code: Delving deeper into SICA's implementation reveals several key design choices that underpin its capabilities. As discussed, the system is built with a modular architecture, incorporating several sub-agents, such as a coding agent, a problem-solver agent, and a reasoning agent. These sub-agents are invoked by the main agent, much like tool calls, serving to decompose complex tasks and efficiently manage context length, especially during those extended meta-improvement iterations.

The project is actively developed and aims to provide a robust framework for those interested in post-training LLMs on tool use and other agentic tasks, with the full code available for further exploration and contribution at the https://github.com/MaximeRobeyns/self_improving_coding_agent/ GitHub repository.

For security, the project strongly emphasizes Docker containerization, meaning the agent runs within a dedicated Docker container. This is a crucial measure, as it provides isolation from the host machine, mitigating risks like inadvertent file system manipulation given the agent's ability to execute shell commands.

To ensure transparency and control, the system features robust observability through an interactive webpage that visualizes events on the event bus and the agent's callgraph. This offers comprehensive insights into the agent's actions, allowing users to inspect individual events, read overseer messages, and collapse sub-agent traces for clearer understanding.

In terms of its core intelligence, the agent framework supports LLM integration from various providers, enabling experimentation with different models to find the best fit for specific tasks. Finally, a critical component is the asynchronous overseer, an LLM that runs concurrently with the main agent. This overseer periodically assesses the agent's behavior for pathological deviations or stagnation and can intervene by sending notifications or even cancelling the agent's execution if necessary. It receives a detailed textual representation of the system's state, including a callgraph and an event stream of LLM messages, tool calls, and responses, which allows it to detect inefficient patterns or repeated work.

A notable challenge in the initial SICA implementation was prompting the LLM-based agent to independently propose novel, innovative, feasible, and engaging modifications during each meta-improvement iteration. This limitation, particularly in

fostering open-ended learning and authentic creativity in LLM agents, remains a key area of investigation in current research.

AlphaEvolve and OpenEvolve

AlphaEvolve is an AI agent developed by Google designed to discover and optimize algorithms. It utilizes a combination of LLMs, specifically Gemini models (Flash and Pro), automated evaluation systems, and an evolutionary algorithm framework. This system aims to advance both theoretical mathematics and practical computing applications.

AlphaEvolve employs an ensemble of Gemini models. Flash is used for generating a wide range of initial algorithm proposals, while Pro provides more in-depth analysis and refinement. Proposed algorithms are then automatically evaluated and scored based on predefined criteria. This evaluation provides feedback that is used to iteratively improve the solutions, leading to optimized and novel algorithms.

In practical computing, AlphaEvolve has been deployed within Google's infrastructure. It has demonstrated improvements in data center scheduling, resulting in a 0.7% reduction in global compute resource usage. It has also contributed to hardware design by suggesting optimizations for Verilog code in upcoming Tensor Processing Units (TPUs). Furthermore, AlphaEvolve has accelerated AI performance, including a 23% speed improvement in a core kernel of the Gemini architecture and up to 32.5% optimization of low-level GPU instructions for FlashAttention.

In the realm of fundamental research, AlphaEvolve has contributed to the discovery of new algorithms for matrix multiplication, including a method for 4x4 complex-valued matrices that uses 48 scalar multiplications, surpassing previously known solutions. In broader mathematical research, it has rediscovered existing state-of-the-art solutions to over 50 open problems in 75% of cases and improved upon existing solutions in 20% of cases, with examples including advancements in the kissing number problem.

OpenEvolve is an evolutionary coding agent that leverages LLMs (see Fig.3) to iteratively optimize code. It orchestrates a pipeline of LLM-driven code generation, evaluation, and selection to continuously enhance programs for a wide range of tasks. A key aspect of OpenEvolve is its capability to evolve entire code files, rather than being limited to single functions. The agent is designed for versatility, offering support for multiple programming languages and compatibility with OpenAI-compatible APIs

for any LLM. Furthermore, it incorporates multi-objective optimization, allows for flexible prompt engineering, and is capable of distributed evaluation to efficiently handle complex coding challenges.

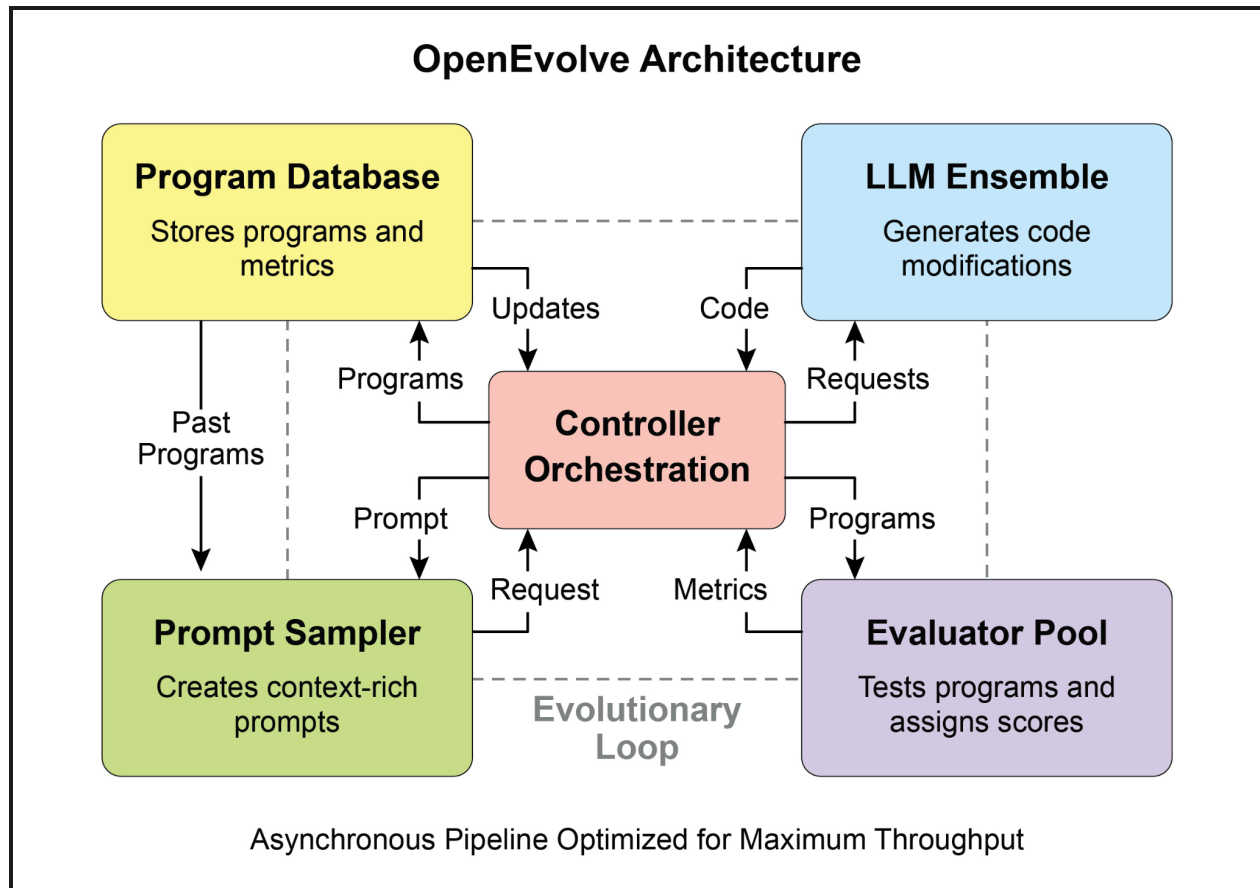


Fig. 3: The OpenEvolve internal architecture is managed by a controller. This controller orchestrates several key components: the program sampler, Program Database, Evaluator Pool, and LLM Ensembles. Its primary function is to facilitate their learning and adaptation processes to enhance code quality.

This code snippet uses the OpenEvolve library to perform evolutionary optimization on a program. It initializes the OpenEvolve system with paths to an initial program, an evaluation file, and a configuration file. The `evolve.run(iterations=1000)` line starts the evolutionary process, running for 1000 iterations to find an improved version of the program. Finally, it prints the metrics of the best program found during the evolution, formatted to four decimal places.

```
from openevolve import OpenEvolve
```

```
# Initialize the system
evolve = OpenEvolve(
    initial_program_path="path/to/initial_program.py",
    evaluation_file="path/to/evaluator.py",
    config_path="path/to/config.yaml"
)

# Run the evolution
best_program = await evolve.run(iterations=1000)
print(f"Best program metrics:")
for name, value in best_program.metrics.items():
    print(f"    {name}: {value:.4f}")
```

At a Glance

What: AI agents often operate in dynamic and unpredictable environments where pre-programmed logic is insufficient. Their performance can degrade when faced with novel situations not anticipated during their initial design. Without the ability to learn from experience, agents cannot optimize their strategies or personalize their interactions over time. This rigidity limits their effectiveness and prevents them from achieving true autonomy in complex, real-world scenarios.

Why: The standardized solution is to integrate learning and adaptation mechanisms, transforming static agents into dynamic, evolving systems. This allows an agent to autonomously refine its knowledge and behaviors based on new data and interactions. Agentic systems can use various methods, from reinforcement learning to more advanced techniques like self-modification, as seen in the Self-Improving Coding Agent (SICA). Advanced systems like Google's AlphaEvolve leverage LLMs and evolutionary algorithms to discover entirely new and more efficient solutions to complex problems. By continuously learning, agents can master new tasks, enhance their performance, and adapt to changing conditions without requiring constant manual reprogramming.

Rule of thumb: Use this pattern when building agents that must operate in dynamic, uncertain, or evolving environments. It is essential for applications requiring personalization, continuous performance improvement, and the ability to handle novel situations autonomously.

Visual summary

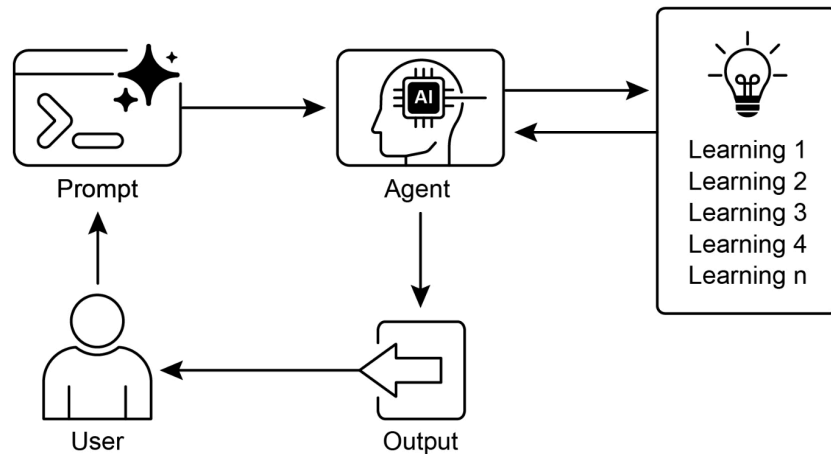


Fig.4: Learning and adapting pattern

Key Takeaways

- Learning and Adaptation are about agents getting better at what they do and handling new situations by using their experiences.
- "Adaptation" is the visible change in an agent's behavior or knowledge that comes from learning.
- SICA, the Self-Improving Coding Agent, self-improves by modifying its code based on past performance. This led to tools like the Smart Editor and AST Symbol Locator.
- Having specialized "sub-agents" and an "overseer" helps these self-improving systems manage big tasks and stay on track.
- The way an LLM's "context window" is set up (with system prompts, core prompts, and assistant messages) is super important for how efficiently agents work.
- This pattern is vital for agents that need to operate in environments that are always changing, uncertain, or require a personal touch.

- Building agents that learn often means hooking them up with machine learning tools and managing how data flows.
- An agent system, equipped with basic coding tools, can autonomously edit itself, and thereby improve its performance on benchmark tasks
- AlphaEvolve is Google's AI agent that leverages LLMs and an evolutionary framework to autonomously discover and optimize algorithms, significantly enhancing both fundamental research and practical computing applications..

Conclusion

This chapter examines the crucial roles of learning and adaptation in Artificial Intelligence. AI agents enhance their performance through continuous data acquisition and experience. The Self-Improving Coding Agent (SICA) exemplifies this by autonomously improving its capabilities through code modifications.

We have reviewed the fundamental components of agentic AI, including architecture, applications, planning, multi-agent collaboration, memory management, and learning and adaptation. Learning principles are particularly vital for coordinated improvement in multi-agent systems. To achieve this, tuning data must accurately reflect the complete interaction trajectory, capturing the individual inputs and outputs of each participating agent.

These elements contribute to significant advancements, such as Google's AlphaEvolve. This AI system independently discovers and refines algorithms by LLMs, automated assessment, and an evolutionary approach, driving progress in scientific research and computational techniques. Such patterns can be combined to construct sophisticated AI systems. Developments like AlphaEvolve demonstrate that autonomous algorithmic discovery and optimization by AI agents are attainable.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
3. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
4. Proximal Policy Optimization Algorithms by John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. You can find it on arXiv: <https://arxiv.org/abs/1707.06347>

5. Robeyns, M., Aitchison, L., & Szummer, M. (2025). *A Self-Improving Coding Agent*. arXiv:2504.15228v2. <https://arxiv.org/pdf/2504.15228>
https://github.com/MaximeRobeyns/self_improving_coding_agent
6. AlphaEvolve blog,
<https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>
7. OpenEvolve, <https://github.com/codelion/openevolve>

Chapter 10: Model Context Protocol

To enable LLMs to function effectively as agents, their capabilities must extend beyond multimodal generation. Interaction with the external environment is necessary, including access to current data, utilization of external software, and execution of specific operational tasks. The Model Context Protocol (MCP) addresses this need by providing a standardized interface for LLMs to interface with external resources. This protocol serves as a key mechanism to facilitate consistent and predictable integration.

MCP Pattern Overview

Imagine a universal adapter that allows any LLM to plug into any external system, database, or tool without a custom integration for each one. That's essentially what the Model Context Protocol (MCP) is. It's an open standard designed to standardize how LLMs like Gemini, OpenAI's GPT models, Mixtral, and Claude communicate with external applications, data sources, and tools. Think of it as a universal connection mechanism that simplifies how LLMs obtain context, execute actions, and interact with various systems.

MCP operates on a client-server architecture. It defines how different elements—data (referred to as resources), interactive templates (which are essentially prompts), and actionable functions (known as tools)—are exposed by an MCP server. These are then consumed by an MCP client, which could be an LLM host application or an AI agent itself. This standardized approach dramatically reduces the complexity of integrating LLMs into diverse operational environments.

However, MCP is a contract for an "agentic interface," and its effectiveness depends heavily on the design of the underlying APIs it exposes. There is a risk that developers simply wrap pre-existing, legacy APIs without modification, which can be suboptimal for an agent. For example, if a ticketing system's API only allows retrieving full ticket details one by one, an agent asked to summarize high-priority tickets will be slow and inaccurate at high volumes. To be truly effective, the underlying API should be improved with deterministic features like filtering and sorting to help the non-deterministic agent work efficiently. This highlights that agents do not magically replace deterministic workflows; they often require stronger deterministic support to succeed.

Furthermore, MCP can wrap an API whose input or output is still not inherently understandable by the agent. An API is only useful if its data format is agent-friendly, a guarantee that MCP itself does not enforce. For instance, creating an MCP server for a document store that returns files as PDFs is mostly useless if the consuming agent cannot parse PDF content. The better approach would be to first create an API that returns a textual version of the document, such as Markdown, which the agent can actually read and process. This demonstrates that developers must consider not just the connection, but the nature of the data being exchanged to ensure true compatibility.

MCP vs. Tool Function Calling

The Model Context Protocol (MCP) and tool function calling are distinct mechanisms that enable LLMs to interact with external capabilities (including tools) and execute actions. While both serve to extend LLM capabilities beyond text generation, they differ in their approach and level of abstraction.

Tool function calling can be thought of as a direct request from an LLM to a specific, pre-defined tool or function. Note that in this context we use the words "tool" and "function" interchangeably. This interaction is characterized by a one-to-one communication model, where the LLM formats a request based on its understanding of a user's intent requiring external action. The application code then executes this request and returns the result to the LLM. This process is often proprietary and varies across different LLM providers.

In contrast, the Model Context Protocol (MCP) operates as a standardized interface for LLMs to discover, communicate with, and utilize external capabilities. It functions as an open protocol that facilitates interaction with a wide range of tools and systems, aiming to establish an ecosystem where any compliant tool can be accessed by any compliant LLM. This fosters interoperability, composability and reusability across different systems and implementations. By adopting a federated model, we significantly improve interoperability and unlock the value of existing assets. This strategy allows us to bring disparate and legacy services into a modern ecosystem simply by wrapping them in an MCP-compliant interface. These services continue to operate independently, but can now be composed into new applications and workflows, with their collaboration orchestrated by LLMs. This fosters agility and reusability without requiring costly rewrites of foundational systems.

Here's a breakdown of the fundamental distinctions between MCP and tool function calling:

Feature	Tool Function Calling	Model Context Protocol (MCP)
Standardization	Proprietary and vendor-specific. The format and implementation differ across LLM providers.	An open, standardized protocol, promoting interoperability between different LLMs and tools.
Scope	A direct mechanism for an LLM to request the execution of a specific, predefined function.	A broader framework for how LLMs and external tools discover and communicate with each other.
Architecture	A one-to-one interaction between the LLM and the application's tool-handling logic.	A client-server architecture where LLM-powered applications (clients) can connect to and utilize various MCP servers (tools).
Discovery	The LLM is explicitly told which tools are available within the context of a specific conversation.	Enables dynamic discovery of available tools. An MCP client can query a server to see what capabilities it offers.
Reusability	Tool integrations are often tightly coupled with the specific application and LLM being used.	Promotes the development of reusable, standalone "MCP servers" that can be accessed by any compliant application.

Think of tool function calling as giving an AI a specific set of custom-built tools, like a particular wrench and screwdriver. This is efficient for a workshop with a fixed set of tasks. MCP (Model Context Protocol), on the other hand, is like creating a universal, standardized power outlet system. It doesn't provide the tools itself, but it allows any compliant tool from any manufacturer to plug in and work, enabling a dynamic and ever-expanding workshop.

In short, function calling provides direct access to a few specific functions, while MCP is the standardized communication framework that lets LLMs discover and use a vast range of external resources. For simple applications, specific tools are enough; for complex, interconnected AI systems that need to adapt, a universal standard like MCP is essential.

Additional considerations for MCP

While MCP presents a powerful framework, a thorough evaluation requires considering several crucial aspects that influence its suitability for a given use case. Let's see some aspects in more details:

- **Tool vs. Resource vs. Prompt:** It's important to understand the specific roles of these components. A resource is static data (e.g., a PDF file, a database record). A tool is an executable function that performs an action (e.g., sending an email, querying an API). A prompt is a template that guides the LLM in how to interact with a resource or tool, ensuring the interaction is structured and effective.
- **Discoverability:** A key advantage of MCP is that an MCP client can dynamically query a server to learn what tools and resources it offers. This "just-in-time" discovery mechanism is powerful for agents that need to adapt to new capabilities without being redeployed.
- **Security:** Exposing tools and data via any protocol requires robust security measures. An MCP implementation must include authentication and authorization to control which clients can access which servers and what specific actions they are permitted to perform.
- **Implementation:** While MCP is an open standard, its implementation can be complex. However, providers are beginning to simplify this process. For example, some model providers like Anthropic or FastMCP offer SDKs that abstract away much of the boilerplate code, making it easier for developers to create and connect MCP clients and servers.
- **Error Handling:** A comprehensive error-handling strategy is critical. The protocol must define how errors (e.g., tool execution failure, unavailable server, invalid request) are communicated back to the LLM so it can understand the failure and potentially try an alternative approach.
- **Local vs. Remote Server:** MCP servers can be deployed locally on the same machine as the agent or remotely on a different server. A local server might be chosen for speed and security with sensitive data, while a remote server

architecture allows for shared, scalable access to common tools across an organization.

- **On-demand vs. Batch:** MCP can support both on-demand, interactive sessions and larger-scale batch processing. The choice depends on the application, from a real-time conversational agent needing immediate tool access to a data analysis pipeline that processes records in batches.
- **Transportation Mechanism:** The protocol also defines the underlying transport layers for communication. For local interactions, it uses JSON-RPC over STDIO (standard input/output) for efficient inter-process communication. For remote connections, it leverages web-friendly protocols like Streamable HTTP and Server-Sent Events (SSE) to enable persistent and efficient client-server communication.

The Model Context Protocol uses a client-server model to standardize information flow. Understanding component interaction is key to MCP's advanced agentic behavior:

1. **Large Language Model (LLM):** The core intelligence. It processes user requests, formulates plans, and decides when it needs to access external information or perform an action.
2. **MCP Client:** This is an application or wrapper around the LLM. It acts as the intermediary, translating the LLM's intent into a formal request that conforms to the MCP standard. It is responsible for discovering, connecting to, and communicating with MCP Servers.
3. **MCP Server:** This is the gateway to the external world. It exposes a set of tools, resources, and prompts to any authorized MCP Client. Each server is typically responsible for a specific domain, such as a connection to a company's internal database, an email service, or a public API.
4. **Optional Third-Party (3P) Service:** This represents the actual external tool, application, or data source that the MCP Server manages and exposes. It is the ultimate endpoint that performs the requested action, such as querying a proprietary database, interacting with a SaaS platform, or calling a public weather API.

The interaction flows as follows:

1. **Discovery:** The MCP Client, on behalf of the LLM, queries an MCP Server to ask what capabilities it offers. The server responds with a manifest listing its available tools (e.g., `send_email`), resources (e.g., `customer_database`), and prompts.

2. **Request Formulation:** The LLM determines that it needs to use one of the discovered tools. For instance, it decides to send an email. It formulates a request, specifying the tool to use (`send_email`) and the necessary parameters (recipient, subject, body).
3. **Client Communication:** The MCP Client takes the LLM's formulated request and sends it as a standardized call to the appropriate MCP Server.
4. **Server Execution:** The MCP Server receives the request. It authenticates the client, validates the request, and then executes the specified action by interfacing with the underlying software (e.g., calling the `send()` function of an email API).
5. **Response and Context Update:** After execution, the MCP Server sends a standardized response back to the MCP Client. This response indicates whether the action was successful and includes any relevant output (e.g., a confirmation ID for the sent email). The client then passes this result back to the LLM, updating its context and enabling it to proceed with the next step of its task.

Practical Applications & Use Cases

MCP significantly broadens AI/LLM capabilities, making them more versatile and powerful. Here are nine key use cases:

- **Database Integration:** MCP allows LLMs and agents to seamlessly access and interact with structured data in databases. For instance, using the MCP Toolbox for Databases, an agent can query Google BigQuery datasets to retrieve real-time information, generate reports, or update records, all driven by natural language commands.
- **Generative Media Orchestration:** MCP enables agents to integrate with advanced generative media services. Through MCP Tools for Genmedia Services, an agent can orchestrate workflows involving Google's Imagen for image generation, Google's Veo for video creation, Google's Chirp 3 HD for realistic voices, or Google's Lyria for music composition, allowing for dynamic content creation within AI applications.
- **External API Interaction:** MCP provides a standardized way for LLMs to call and receive responses from any external API. This means an agent can fetch live weather data, pull stock prices, send emails, or interact with CRM systems, extending its capabilities far beyond its core language model.
- **Reasoning-Based Information Extraction:** Leveraging an LLM's strong reasoning skills, MCP facilitates effective, query-dependent information extraction that surpasses conventional search and retrieval systems. Instead of a

traditional search tool returning an entire document, an agent can analyze the text and extract the precise clause, figure, or statement that directly answers a user's complex question.

- **Custom Tool Development:** Developers can build custom tools and expose them via an MCP server (e.g., using FastMCP). This allows specialized internal functions or proprietary systems to be made available to LLMs and other agents in a standardized, easily consumable format, without needing to modify the LLM directly.
- **Standardized LLM-to-Application Communication:** MCP ensures a consistent communication layer between LLMs and the applications they interact with. This reduces integration overhead, promotes interoperability between different LLM providers and host applications, and simplifies the development of complex agentic systems.
- **Complex Workflow Orchestration:** By combining various MCP-exposed tools and data sources, agents can orchestrate highly complex, multi-step workflows. An agent could, for example, retrieve customer data from a database, generate a personalized marketing image, draft a tailored email, and then send it, all by interacting with different MCP services.
- **IoT Device Control:** MCP can facilitate LLM interaction with Internet of Things (IoT) devices. An agent could use MCP to send commands to smart home appliances, industrial sensors, or robotics, enabling natural language control and automation of physical systems.
- **Financial Services Automation:** In financial services, MCP could enable LLMs to interact with various financial data sources, trading platforms, or compliance systems. An agent might analyze market data, execute trades, generate personalized financial advice, or automate regulatory reporting, all while maintaining secure and standardized communication.

In short, the Model Context Protocol (MCP) enables agents to access real-time information from databases, APIs, and web resources. It also allows agents to perform actions like sending emails, updating records, controlling devices, and executing complex tasks by integrating and processing data from various sources. Additionally, MCP supports media generation tools for AI applications.

Hands-On Code Example with ADK

This section outlines how to connect to a local MCP server that provides file system operations, enabling an ADK agent to interact with the local file system.

Agent Setup with MCPToolset

To configure an agent for file system interaction, an `agent.py` file must be created (e.g., at `./adk\_agent\_samples/mcp\_agent/agent.py`). The `MCPToolset` is instantiated within the `tools` list of the `LlmAgent` object. It is crucial to replace `"/path/to/your/folder"` in the `args` list with the absolute path to a directory on the local system that the MCP server can access. This directory will be the root for the file system operations performed by the agent.

```
import os
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset,
StudioServerParameters

# Create a reliable absolute path to a folder named
'mcp_managed_files'
# within the same directory as this agent script.
# This ensures the agent works out-of-the-box for demonstration.
# For production, you would point this to a more persistent and
secure location.
TARGET_FOLDER_PATH =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"mcp_managed_files")

# Ensure the target directory exists before the agent needs it.
os.makedirs(TARGET_FOLDER_PATH, exist_ok=True)

root_agent = LlmAgent(
    model='gemini-2.0-flash',
    name='filesystem_assistant_agent',
    instruction=(
        'Help the user manage their files. You can list files, read
files, and write files. '
        f'You are operating in the following directory:
{TARGET_FOLDER_PATH}'
    ),
    tools=[
        MCPToolset(
            connection_params=StudioServerParameters(
                command='npx',
                args=[
                    "-y", # Argument for npx to auto-confirm install
                    "@modelcontextprotocol/server-filesystem",
                    # This MUST be an absolute path to a folder.
```



```

        TARGET_FOLDER_PATH,
    ],
),
# Optional: You can filter which tools from the MCP server
are exposed.
# For example, to only allow reading:
# tool_filter=['list_directory', 'read_file']
)
],
)

```

`npx` (Node Package Execute), bundled with npm (Node Package Manager) versions 5.2.0 and later, is a utility that enables direct execution of Node.js packages from the npm registry. This eliminates the need for global installation. In essence, `npx` serves as an npm package runner, and it is commonly used to run many community MCP servers, which are distributed as Node.js packages.

Creating an `__init__.py` file is necessary to ensure the `agent.py` file is recognized as part of a discoverable Python package for the Agent Development Kit (ADK). This file should reside in the same directory as [agent.py](#).

```

# ./adk_agent_samples/mcp_agent/__init__.py
from . import agent

```

Certainly, other supported commands are available for use. For example, connecting to python3 can be achieved as follows:

```

connection_params = StdioConnectionParams(
    server_params={
        "command": "python3",
        "args": ["../agent/mcp_server.py"],
        "env": {
            "SERVICE_ACCOUNT_PATH": SERVICE_ACCOUNT_PATH,
            "DRIVE_FOLDER_ID": DRIVE_FOLDER_ID
        }
    }
)

```

UVX, in the context of Python, refers to a command-line tool that utilizes uv to execute commands in a temporary, isolated Python environment. Essentially, it allows you to run Python tools and packages without needing to install them globally or within your project's environment. You can run it via the MCP server.

```
connection_params = StdioConnectionParams(  
    server_params={  
        "command": "uvx",  
        "args": ["mcp-google-sheets@latest"],  
        "env": {  
            "SERVICE_ACCOUNT_PATH": SERVICE_ACCOUNT_PATH,  
            "DRIVE_FOLDER_ID": DRIVE_FOLDER_ID  
        }  
    }  
)
```

Once the MCP Server is created, the next step is to connect to it.

Connecting the MCP Server with ADK Web

To begin, execute 'adk web'. Navigate to the parent directory of mcp_agent (e.g., adk_agent_samples) in your terminal and run:

```
cd ./adk_agent_samples # Or your equivalent parent directory  
adk web
```

Once the ADK Web UI has loaded in your browser, select the 'filesystem_assistant_agent' from the agent menu. Next, experiment with prompts such as:

- "Show me the contents of this folder."
- "Read the 'sample.txt' file." (This assumes 'sample.txt' is located at 'TARGET_FOLDER_PATH'.)
- "What's in 'another_file.md'?"

Creating an MCP Server with FastMCP

FastMCP is a high-level Python framework designed to streamline the development of MCP servers. It provides an abstraction layer that simplifies protocol complexities, allowing developers to focus on core logic.

The library enables rapid definition of tools, resources, and prompts using simple Python decorators. A significant advantage is its automatic schema generation, which intelligently interprets Python function signatures, type hints, and documentation strings to construct necessary AI model interface specifications. This automation minimizes manual configuration and reduces human error.

Beyond basic tool creation, FastMCP facilitates advanced architectural patterns like server composition and proxying. This enables modular development of complex, multi-component systems and seamless integration of existing services into an AI-accessible framework. Additionally, FastMCP includes optimizations for efficient, distributed, and scalable AI-driven applications.

Server setup with FastMCP

To illustrate, consider a basic "greet" tool provided by the server. ADK agents and other MCP clients can interact with this tool using HTTP once it is active.

```
# fastmcp_server.py
# This script demonstrates how to create a simple MCP server using FastMCP.
# It exposes a single tool that generates a greeting.

# 1. Make sure you have FastMCP installed:
# pip install fastmcp
from fastmcp import FastMCP, Client

# Initialize the FastMCP server.
mcp_server = FastMCP()

# Define a simple tool function.
# The `@mcp_server.tool` decorator registers this Python function as an MCP
tool.
# The docstring becomes the tool's description for the LLM.
@mcp_server.tool
def greet(name: str) -> str:
    """
    Generates a personalized greeting.

    Args:
```

```
    name: The name of the person to greet.

Returns:
    A greeting string.
    """
    return f"Hello, {name}! Nice to meet you."

# Or if you want to run it from the script:
if __name__ == "__main__":
    mcp_server.run(
        transport="http",
        host="127.0.0.1",
        port=8000
    )
```

This Python script defines a single function called `greet`, which takes a person's name and returns a personalized greeting. The `@tool()` decorator above this function automatically registers it as a tool that an AI or another program can use. The function's documentation string and type hints are used by FastMCP to tell the Agent how the tool works, what inputs it needs, and what it will return.

When the script is executed, it starts the FastMCP server, which listens for requests on `localhost:8000`. This makes the `greet` function available as a network service. An agent could then be configured to connect to this server and use the `greet` tool to generate greetings as part of a larger task. The server runs continuously until it is manually stopped.

Consuming the FastMCP Server with an ADK Agent

An ADK agent can be set up as an MCP client to use a running FastMCP server. This requires configuring `HttpServerParameters` with the FastMCP server's network address, which is usually `http://localhost:8000`.

A `tool_filter` parameter can be included to restrict the agent's tool usage to specific tools offered by the server, such as `'greet'`. When prompted with a request like "Greet John Doe," the agent's embedded LLM identifies the `'greet'` tool available via MCP, invokes it with the argument "John Doe," and returns the server's response. This process demonstrates the integration of user-defined tools exposed through MCP with an ADK agent.

To establish this configuration, an agent file (e.g., `agent.py` located in `./adk_agent_samples/fastmcp_client_agent/`) is required. This file will instantiate an

ADK agent and use `HttpServerParameters` to establish a connection with the operational FastMCP server.

```
# ./adk_agent_samples/fastmcp_client_agent/agent.py
import os
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset,
HttpServerParameters

# Define the FastMCP server's address.
# Make sure your fastmcp_server.py (defined previously) is running on
this port.
FASTMCP_SERVER_URL = "http://localhost:8000"

root_agent = LlmAgent(
    model='gemini-2.0-flash', # Or your preferred model
    name='fastmcp_greeter_agent',
    instruction='You are a friendly assistant that can greet people by
their name. Use the "greet" tool.',
    tools=[
        MCPToolset(
            connection_params=HttpServerParameters(
                url=FASTMCP_SERVER_URL,
            ),
            # Optional: Filter which tools from the MCP server are
exposed
            # For this example, we're expecting only 'greet'
            tool_filter=['greet']
        )
    ],
)
```

The script defines an Agent named `fastmcp_greeter_agent` that uses a Gemini language model. It's given a specific instruction to act as a friendly assistant whose purpose is to greet people. Crucially, the code equips this agent with a tool to perform its task. It configures an `MCPToolset` to connect to a separate server running on `localhost:8000`, which is expected to be the FastMCP server from the previous example. The agent is specifically granted access to the `greet` tool hosted on that server. In essence, this code sets up the client side of the system, creating an intelligent agent that understands its goal is to greet people and knows exactly which external tool to use to accomplish it.

Creating an `__init__.py` file within the `fastmcp_client_agent` directory is necessary. This ensures the agent is recognized as a discoverable Python package for the ADK.

To begin, open a new terminal and run `python fastmcp_server.py` to start the FastMCP server. Next, go to the parent directory of `fastmcp_client_agent` (for example, `adk_agent_samples`) in your terminal and execute `adk web`. Once the ADK Web UI loads in your browser, select the `fastmcp_greeter_agent` from the agent menu. You can then test it by entering a prompt like "Greet John Doe." The agent will use the `greet` tool on your FastMCP server to create a response.

At a Glance

What: To function as effective agents, LLMs must move beyond simple text generation. They require the ability to interact with the external environment to access current data and utilize external software. Without a standardized communication method, each integration between an LLM and an external tool or data source becomes a custom, complex, and non-reusable effort. This ad-hoc approach hinders scalability and makes building complex, interconnected AI systems difficult and inefficient.

Why: The Model Context Protocol (MCP) offers a standardized solution by acting as a universal interface between LLMs and external systems. It establishes an open, standardized protocol that defines how external capabilities are discovered and used. Operating on a client-server model, MCP allows servers to expose tools, data resources, and interactive prompts to any compliant client. LLM-powered applications act as these clients, dynamically discovering and interacting with available resources in a predictable manner. This standardized approach fosters an ecosystem of interoperable and reusable components, dramatically simplifying the development of complex agentic workflows.

Rule of thumb: Use the Model Context Protocol (MCP) when building complex, scalable, or enterprise-grade agentic systems that need to interact with a diverse and evolving set of external tools, data sources, and APIs. It is ideal when interoperability between different LLMs and tools is a priority, and when agents require the ability to dynamically discover new capabilities without being redeployed. For simpler applications with a fixed and limited number of predefined functions, direct tool function calling may be sufficient.

Visual summary

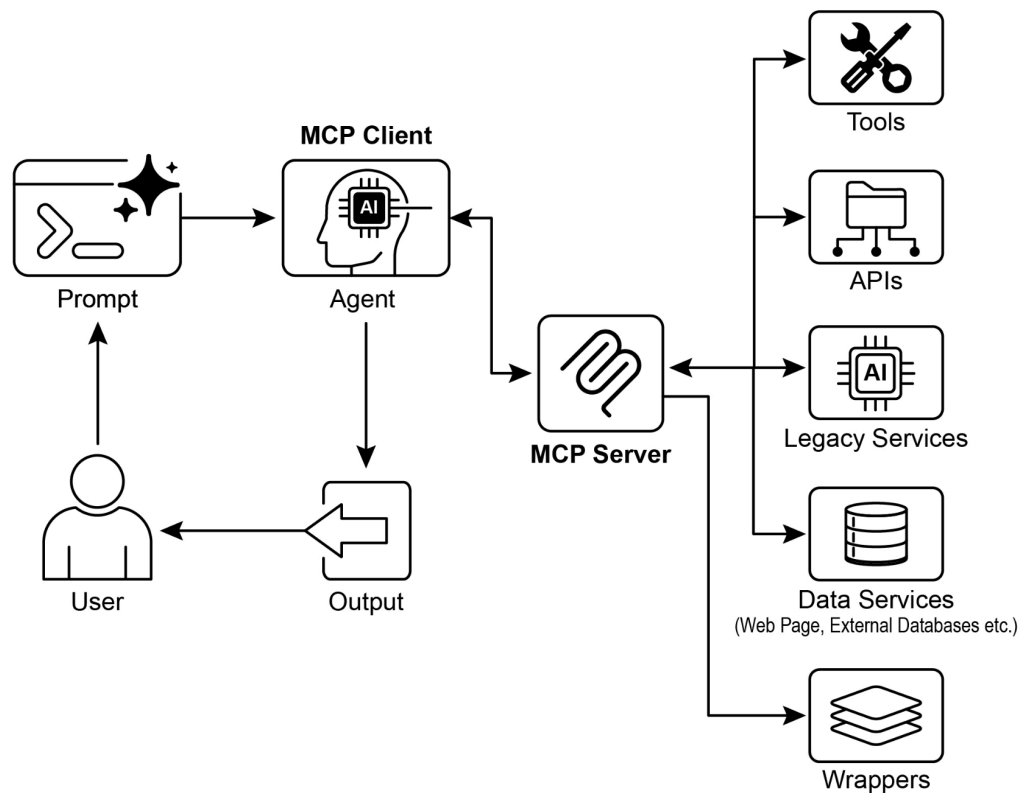


Fig.1: Model Context protocol

Key Takeaways

These are the key takeaways:

- The Model Context Protocol (MCP) is an open standard facilitating standardized communication between LLMs and external applications, data sources, and tools.
- It employs a client-server architecture, defining the methods for exposing and consuming resources, prompts, and tools.
- The Agent Development Kit (ADK) supports both utilizing existing MCP servers and exposing ADK tools via an MCP server.
- FastMCP simplifies the development and management of MCP servers, particularly for exposing tools implemented in Python.
- MCP Tools for Genmedia Services allows agents to integrate with Google Cloud's

- generative media capabilities (Imagen, Veo, Chirp 3 HD, Lyria).
- MCP enables LLMs and agents to interact with real-world systems, access dynamic information, and perform actions beyond text generation.

Conclusion

The Model Context Protocol (MCP) is an open standard that facilitates communication between Large Language Models (LLMs) and external systems. It employs a client-server architecture, enabling LLMs to access resources, utilize prompts, and execute actions through standardized tools. MCP allows LLMs to interact with databases, manage generative media workflows, control IoT devices, and automate financial services. Practical examples demonstrate setting up agents to communicate with MCP servers, including filesystem servers and servers built with FastMCP, illustrating its integration with the Agent Development Kit (ADK). MCP is a key component for developing interactive AI agents that extend beyond basic language capabilities.

References

1. Model Context Protocol (MCP) Documentation. (Latest). *Model Context Protocol (MCP)*. <https://google.github.io/adk-docs/mcp/>
2. FastMCP Documentation. FastMCP. <https://github.com/jlowin/fastmcp>
3. MCP Tools for Genmedia Services. *MCP Tools for Genmedia Services*. <https://google.github.io/adk-docs/mcp/#mcp-servers-for-google-cloud-genmedia>
4. MCP Toolbox for Databases Documentation. (Latest). *MCP Toolbox for Databases*. <https://google.github.io/adk-docs/mcp/databases/>

Chapter 11: Goal Setting and Monitoring

For AI agents to be truly effective and purposeful, they need more than just the ability to process information or use tools; they need a clear sense of direction and a way to know if they're actually succeeding. This is where the Goal Setting and Monitoring pattern comes into play. It's about giving agents specific objectives to work towards and equipping them with the means to track their progress and determine if those objectives have been met.

Goal Setting and Monitoring Pattern Overview

Think about planning a trip. You don't just spontaneously appear at your destination. You decide where you want to go (the goal state), figure out where you are starting from (the initial state), consider available options (transportation, routes, budget), and then map out a sequence of steps: book tickets, pack bags, travel to the airport/station, board the transport, arrive, find accommodation, etc. This step-by-step process, often considering dependencies and constraints, is fundamentally what we mean by planning in agentic systems.

In the context of AI agents, planning typically involves an agent taking a high-level objective and autonomously, or semi-autonomously, generating a series of intermediate steps or sub-goals. These steps can then be executed sequentially or in a more complex flow, potentially involving other patterns like tool use, routing, or multi-agent collaboration. The planning mechanism might involve sophisticated search algorithms, logical reasoning, or increasingly, leveraging the capabilities of large language models (LLMs) to generate plausible and effective plans based on their training data and understanding of tasks.

A good planning capability allows agents to tackle problems that aren't simple, single-step queries. It enables them to handle multi-faceted requests, adapt to changing circumstances by replanning, and orchestrate complex workflows. It's a foundational pattern that underpins many advanced agentic behaviors, turning a simple reactive system into one that can proactively work towards a defined objective.

Practical Applications & Use Cases

The Goal Setting and Monitoring pattern is essential for building agents that can operate autonomously and reliably in complex, real-world scenarios. Here are some practical applications:

- **Customer Support Automation:** An agent's goal might be to "resolve customer's billing inquiry." It monitors the conversation, checks database entries, and uses tools to adjust billing. Success is monitored by confirming the billing change and receiving positive customer feedback. If the issue isn't resolved, it escalates.
- **Personalized Learning Systems:** A learning agent might have the goal to "improve students' understanding of algebra." It monitors the student's progress on exercises, adapts teaching materials, and tracks performance metrics like accuracy and completion time, adjusting its approach if the student struggles.
- **Project Management Assistants:** An agent could be tasked with "ensuring project milestone X is completed by Y date." It monitors task statuses, team communications, and resource availability, flagging delays and suggesting corrective actions if the goal is at risk.
- **Automated Trading Bots:** A trading agent's goal might be to "maximize portfolio gains while staying within risk tolerance." It continuously monitors market data, its current portfolio value, and risk indicators, executing trades when conditions align with its goals and adjusting strategy if risk thresholds are breached.
- **Robotics and Autonomous Vehicles:** An autonomous vehicle's primary goal is "safely transport passengers from A to B." It constantly monitors its environment (other vehicles, pedestrians, traffic signals), its own state (speed, fuel), and its progress along the planned route, adapting its driving behavior to achieve the goal safely and efficiently.
- **Content Moderation:** An agent's goal could be to "identify and remove harmful content from platform X." It monitors incoming content, applies classification models, and tracks metrics like false positives/negatives, adjusting its filtering criteria or escalating ambiguous cases to human reviewers.

This pattern is fundamental for agents that need to operate reliably, achieve specific outcomes, and adapt to dynamic conditions, providing the necessary framework for intelligent self-management.

Hands-On Code Example

To illustrate the Goal Setting and Monitoring pattern, we have an example using LangChain and OpenAI APIs. This Python script outlines an autonomous AI agent engineered to generate and refine Python code. Its core function is to produce solutions for specified problems, ensuring adherence to user-defined quality benchmarks.

It employs a "goal-setting and monitoring" pattern where it doesn't just generate code once, but enters into an iterative cycle of creation, self-evaluation, and improvement.

The agent's success is measured by its own AI-driven judgment on whether the generated code successfully meets the initial objectives. The ultimate output is a polished, commented, and ready-to-use Python file that represents the culmination of this refinement process.

Dependencies:

```
pip install langchain_openai openai python-dotenv
.env file with key in OPENAI_API_KEY
```

You can best understand this script by imagining it as an autonomous AI programmer assigned to a project (see Fig. 1). The process begins when you hand the AI a detailed project brief, which is the specific coding problem it needs to solve.

```
# MIT License
# Copyright (c) 2025 Mahtab Syed
# https://www.linkedin.com/in/mahtabsyed/

"""
Hands-On Code Example - Iteration 2
- To illustrate the Goal Setting and Monitoring pattern, we have an
example using LangChain and OpenAI APIs:

Objective: Build an AI Agent which can write code for a specified
use case based on specified goals:
- Accepts a coding problem (use case) in code or can be as input.
- Accepts a list of goals (e.g., "simple", "tested", "handles edge
cases") in code or can be input.
- Uses an LLM (like GPT-4o) to generate and refine Python code
until the goals are met. (I am using max 5 iterations, this could
be based on a set goal as well)
- To check if we have met our goals I am asking the LLM to judge
this and answer just True or False which makes it easier to stop
the iterations.
- Saves the final code in a .py file with a clean filename and a
header comment.
"""

import os
import random
import re
from pathlib import Path
from langchain_openai import ChatOpenAI
```

```

from dotenv import load_dotenv, find_dotenv

# 🗝️ Load environment variables
_ = load_dotenv(find_dotenv())
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
if not OPENAI_API_KEY:
    raise EnvironmentError("❌ Please set the OPENAI_API_KEY
environment variable.")

# ✅ Initialize OpenAI model
print("📡 Initializing OpenAI LLM (gpt-4o)...")
llm = ChatOpenAI(
    model="gpt-4o", # If you dont have access to got-4o use other
OpenAI LLMs
    temperature=0.3,
    openai_api_key=OPENAI_API_KEY,
)

# --- Utility Functions ---

def generate_prompt(
    use_case: str, goals: list[str], previous_code: str = "",
feedback: str = ""
) -> str:
    print("📝 Constructing prompt for code generation...")
    base_prompt = f"""
You are an AI coding agent. Your job is to write Python code based
on the following use case:

Use Case: {use_case}

Your goals are:
{chr(10).join(f"- {g.strip()}" for g in goals)}
"""
    if previous_code:
        print("🔄 Adding previous code to the prompt for
refinement.")
        base_prompt += f"\nPreviously generated
code:\n{previous_code}"
    if feedback:
        print("📋 Including feedback for revision.")
        base_prompt += f"\nFeedback on previous
version:\n{feedback}\n"

    base_prompt += "\nPlease return only the revised Python code. Do
not include comments or explanations outside the code."
    return base_prompt

```

```

def get_code_feedback(code: str, goals: list[str]) -> str:
    print("🔍 Evaluating code against the goals...")
    feedback_prompt = f"""
You are a Python code reviewer. A code snippet is shown below.
Based on the following goals:

{chr(10).join(f"- {g.strip()}" for g in goals)}

Please critique this code and identify if the goals are met.
Mention if improvements are needed for clarity, simplicity,
correctness, edge case handling, or test coverage.

Code:
{code}
"""
    return llm.invoke(feedback_prompt)

def goals_met(feedback_text: str, goals: list[str]) -> bool:
    """
    Uses the LLM to evaluate whether the goals have been met based
    on the feedback text.
    Returns True or False (parsed from LLM output).
    """
    review_prompt = f"""
You are an AI reviewer.

Here are the goals:
{chr(10).join(f"- {g.strip()}" for g in goals)}

Here is the feedback on the code:
\"\"\"
{feedback_text}
\"\"\"

Based on the feedback above, have the goals been met?

Respond with only one word: True or False.
"""
    response = llm.invoke(review_prompt).content.strip().lower()
    return response == "true"

def clean_code_block(code: str) -> str:
    lines = code.strip().splitlines()
    if lines and lines[0].strip().startswith("`"):
        lines = lines[1:]
    if lines and lines[-1].strip() == "`":

```

```

        lines = lines[:-1]
        return "\n".join(lines).strip()

def add_comment_header(code: str, use_case: str) -> str:
    comment = f"# This Python program implements the following use
case:\n# {use_case.strip()}\n"
    return comment + "\n" + code

def to_snake_case(text: str) -> str:
    text = re.sub(r"^[a-zA-Z0-9 ]", "", text)
    return re.sub(r"\s+", "_", text.strip().lower())

def save_code_to_file(code: str, use_case: str) -> str:
    print("💾 Saving final code to file...")

    summary_prompt = (
        f"Summarize the following use case into a single lowercase
word or phrase, "
        f"no more than 10 characters, suitable for a Python
filename:\n\n{use_case}"
    )
    raw_summary = llm.invoke(summary_prompt).content.strip()
    short_name = re.sub(r"^[a-zA-Z0-9_]", "", raw_summary.replace("
", "_").lower())[:10]

    random_suffix = str(random.randint(1000, 9999))
    filename = f"{short_name}_{random_suffix}.py"
    filepath = Path.cwd() / filename

    with open(filepath, "w") as f:
        f.write(code)

    print(f"✅ Code saved to: {filepath}")
    return str(filepath)

# --- Main Agent Function ---

def run_code_agent(use_case: str, goals_input: str, max_iterations:
int = 5) -> str:
    goals = [g.strip() for g in goals_input.split(",")]

    print(f"\n🎯 Use Case: {use_case}")
    print("🎯 Goals:")
    for g in goals:
        print(f"  - {g}")

    previous_code = ""

```

```

feedback = ""

for i in range(max_iterations):
    print(f"\n=== 🔄 Iteration {i + 1} of {max_iterations} ===")
    prompt = generate_prompt(use_case, goals, previous_code,
feedback if isinstance(feedback, str) else feedback.content)

    print("🚧 Generating code...")
    code_response = llm.invoke(prompt)
    raw_code = code_response.content.strip()
    code = clean_code_block(raw_code)
    print("\n📄 Generated Code:\n" + "-" * 50 + f"\n{code}\n" +
    "-" * 50)

    print("\n📤 Submitting code for feedback review...")
    feedback = get_code_feedback(code, goals)
    feedback_text = feedback.content.strip()
    print("\n📖 Feedback Received:\n" + "-" * 50 +
f"\n{feedback_text}\n" + "-" * 50)

    if goals_met(feedback_text, goals):
        print("✅ LLM confirms goals are met. Stopping
iteration.")
        break

    print("🔧 Goals not fully met. Preparing for next
iteration...")
    previous_code = code

    final_code = add_comment_header(code, use_case)
    return save_code_to_file(final_code, use_case)

# --- CLI Test Run ---

if __name__ == "__main__":
    print("\n🧠 Welcome to the AI Code Generation Agent")

    # Example 1
    use_case_input = "Write code to find BinaryGap of a given
positive integer"
    goals_input = "Code simple to understand, Functionally correct,
Handles comprehensive edge cases, Takes positive integer input
only, prints the results with few examples"
    run_code_agent(use_case_input, goals_input)

    # Example 2
    # use_case_input = "Write code to count the number of files in

```

```
current directory and all its nested sub directories, and print the
total count"
    # goals_input = (
    #     "Code simple to understand, Functionally correct, Handles
comprehensive edge cases, Ignore recommendations for performance,
Ignore recommendations for test suite use like unittest or pytest"
    # )
    # run_code_agent(use_case_input, goals_input)

# Example 3
# use_case_input = "Write code which takes a command line input
of a word doc or docx file and opens it and counts the number of
words, and characters in it and prints all"
# goals_input = "Code simple to understand, Functionally
correct, Handles edge cases"
# run_code_agent(use_case_input, goals_input)
```

Along with this brief, you provide a strict quality checklist, which represents the objectives the final code must meet—criteria like "the solution must be simple," "it must be functionally correct," or "it needs to handle unexpected edge cases."

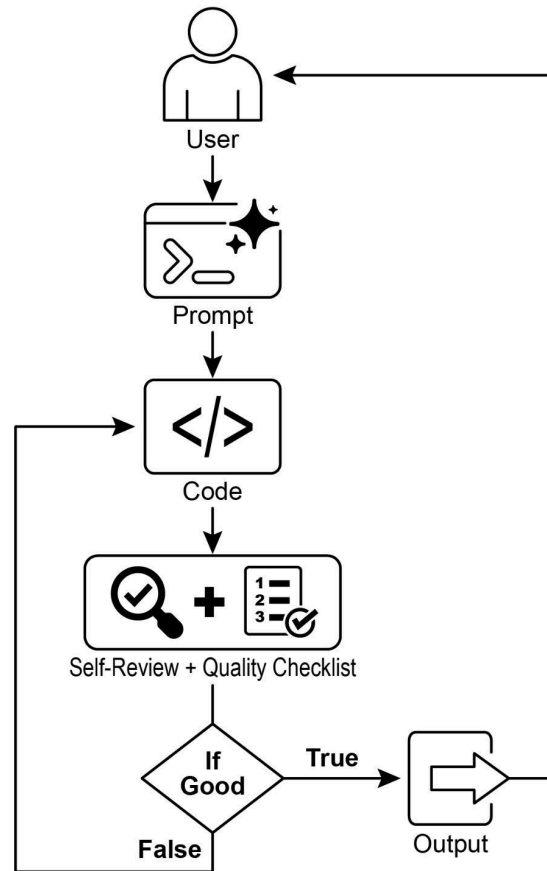


Fig.1: Goal Setting and Monitor example

With this assignment in hand, the AI programmer gets to work and produces its first draft of the code. However, instead of immediately submitting this initial version, it pauses to perform a crucial step: a rigorous self-review. It meticulously compares its own creation against every item on the quality checklist you provided, acting as its own quality assurance inspector. After this inspection, it renders a simple, unbiased verdict on its own progress: "True" if the work meets all standards, or "False" if it falls short.

If the verdict is "False," the AI doesn't give up. It enters a thoughtful revision phase, using the insights from its self-critique to pinpoint the weaknesses and intelligently rewrite the code. This cycle of drafting, self-reviewing, and refining continues, with each iteration aiming to get closer to the goals. This process repeats until the AI finally achieves a "True" status by satisfying every requirement, or until it reaches a predefined limit of attempts, much like a developer working against a deadline. Once

the code passes this final inspection, the script packages the polished solution, adding helpful comments and saving it to a clean, new Python file, ready for use.

Caveats and Considerations: It is important to note that this is an exemplary illustration and not production-ready code. For real-world applications, several factors must be taken into account. An LLM may not fully grasp the intended meaning of a goal and might incorrectly assess its performance as successful. Even if the goal is well understood, the model may hallucinate. When the same LLM is responsible for both writing the code and judging its quality, it may have a harder time discovering it is going in the wrong direction.

Ultimately, LLMs do not produce flawless code by magic; you still need to run and test the produced code. Furthermore, the "monitoring" in the simple example is basic and creates a potential risk of the process running forever.

```
Act as an expert code reviewer with a deep commitment to producing
clean, correct, and simple code. Your core mission is to eliminate
code "hallucinations" by ensuring every suggestion is grounded in
reality and best practices.
```

```
When I provide you with a code snippet, I want you to:
```

```
-- Identify and Correct Errors: Point out any logical flaws, bugs, or
potential runtime errors.
```

```
-- Simplify and Refactor: Suggest changes that make the code more
readable, efficient, and maintainable without sacrificing
correctness.
```

```
-- Provide Clear Explanations: For every suggested change, explain
why it is an improvement, referencing principles of clean code,
performance, or security.
```

```
-- Offer Corrected Code: Show the "before" and "after" of your
suggested changes so the improvement is clear.
```

```
Your feedback should be direct, constructive, and always aimed at
improving the quality of the code.
```

A more robust approach involves separating these concerns by giving specific roles to a crew of agents. For instance, I have built a personal crew of AI agents using Gemini where each has a specific role:

- The Peer Programmer: Helps write and brainstorm code.
- The Code Reviewer: Catches errors and suggests improvements.
- The Documenter: Generates clear and concise documentation.
- The Test Writer: Creates comprehensive unit tests.
- The Prompt Refiner: Optimizes interactions with the AI.

In this multi-agent system, the Code Reviewer, acting as a separate entity from the programmer agent, has a prompt similar to the judge in the example, which significantly improves objective evaluation. This structure naturally leads to better practices, as the Test Writer agent can fulfill the need to write unit tests for the code produced by the Peer Programmer.

I leave to the interested reader the task of adding these more sophisticated controls and making the code closer to production-ready.

At a Glance

What: AI agents often lack a clear direction, preventing them from acting with purpose beyond simple, reactive tasks. Without defined objectives, they cannot independently tackle complex, multi-step problems or orchestrate sophisticated workflows. Furthermore, there is no inherent mechanism for them to determine if their actions are leading to a successful outcome. This limits their autonomy and prevents them from being truly effective in dynamic, real-world scenarios where mere task execution is insufficient.

Why: The Goal Setting and Monitoring pattern provides a standardized solution by embedding a sense of purpose and self-assessment into agentic systems. It involves explicitly defining clear, measurable objectives for the agent to achieve. Concurrently, it establishes a monitoring mechanism that continuously tracks the agent's progress and the state of its environment against these goals. This creates a crucial feedback loop, enabling the agent to assess its performance, correct its course, and adapt its plan if it deviates from the path to success. By implementing this pattern, developers can transform simple reactive agents into proactive, goal-oriented systems capable of autonomous and reliable operation.

Rule of thumb: Use this pattern when an AI agent must autonomously execute a multi-step task, adapt to dynamic conditions, and reliably achieve a specific, high-level objective without constant human intervention.

Visual summary:

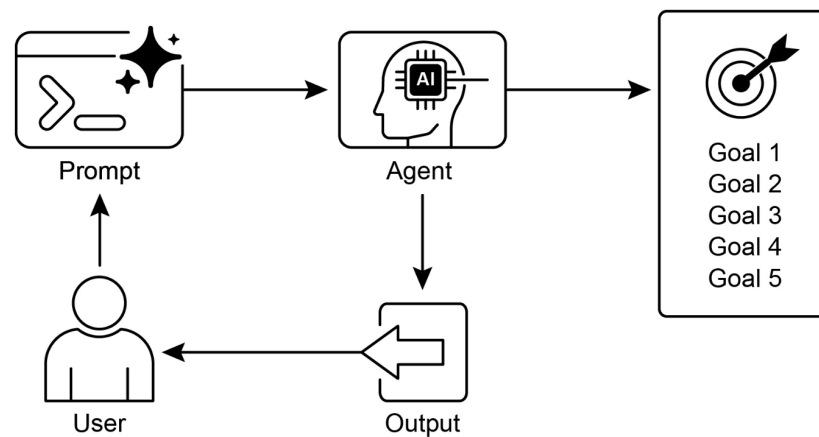


Fig.2: Goal design patterns

Key takeaways

Key takeaways include:

- Goal Setting and Monitoring equips agents with purpose and mechanisms to track progress.
- Goals should be specific, measurable, achievable, relevant, and time-bound (SMART).
- Clearly defining metrics and success criteria is essential for effective monitoring.
- Monitoring involves observing agent actions, environmental states, and tool outputs.
- Feedback loops from monitoring allow agents to adapt, revise plans, or escalate issues.
- In Google's ADK, goals are often conveyed through agent instructions, with

monitoring accomplished through state management and tool interactions.

Conclusion

This chapter focused on the crucial paradigm of Goal Setting and Monitoring. I highlighted how this concept transforms AI agents from merely reactive systems into proactive, goal-driven entities. The text emphasized the importance of defining clear, measurable objectives and establishing rigorous monitoring procedures to track progress. Practical applications demonstrated how this paradigm supports reliable autonomous operation across various domains, including customer service and robotics. A conceptual coding example illustrates the implementation of these principles within a structured framework, using agent directives and state management to guide and evaluate an agent's achievement of its specified goals. Ultimately, equipping agents with the ability to formulate and oversee goals is a fundamental step toward building truly intelligent and accountable AI systems.

References

1. SMART Goals Framework. https://en.wikipedia.org/wiki/SMART_criteria

Chapter 12: Exception Handling and Recovery

For AI agents to operate reliably in diverse real-world environments, they must be able to manage unforeseen situations, errors, and malfunctions. Just as humans adapt to unexpected obstacles, intelligent agents need robust systems to detect problems, initiate recovery procedures, or at least ensure controlled failure. This essential requirement forms the basis of the Exception Handling and Recovery pattern.

This pattern focuses on developing exceptionally durable and resilient agents that can maintain uninterrupted functionality and operational integrity despite various difficulties and anomalies. It emphasizes the importance of both proactive preparation and reactive strategies to ensure continuous operation, even when facing challenges. This adaptability is critical for agents to function successfully in complex and unpredictable settings, ultimately boosting their overall effectiveness and trustworthiness.

The capacity to handle unexpected events ensures these AI systems are not only intelligent but also stable and reliable, which fosters greater confidence in their deployment and performance. Integrating comprehensive monitoring and diagnostic tools further strengthens an agent's ability to quickly identify and address issues, preventing potential disruptions and ensuring smoother operation in evolving conditions. These advanced systems are crucial for maintaining the integrity and efficiency of AI operations, reinforcing their ability to manage complexity and unpredictability.

This pattern may sometimes be used with reflection. For example, if an initial attempt fails and raises an exception, a reflective process can analyze the failure and reattempt the task with a refined approach, such as an improved prompt, to resolve the error.

Exception Handling and Recovery Pattern Overview

The Exception Handling and Recovery pattern addresses the need for AI agents to manage operational failures. This pattern involves anticipating potential issues, such as tool errors or service unavailability, and developing strategies to mitigate them. These strategies may include error logging, retries, fallbacks, graceful degradation,

and notifications. Additionally, the pattern emphasizes recovery mechanisms like state rollback, diagnosis, self-correction, and escalation, to restore agents to stable operation. Implementing this pattern enhances the reliability and robustness of AI agents, allowing them to function in unpredictable environments. Examples of practical applications include chatbots managing database errors, trading bots handling financial errors, and smart home agents addressing device malfunctions. The pattern ensures that agents can continue to operate effectively despite encountering complexities and failures.

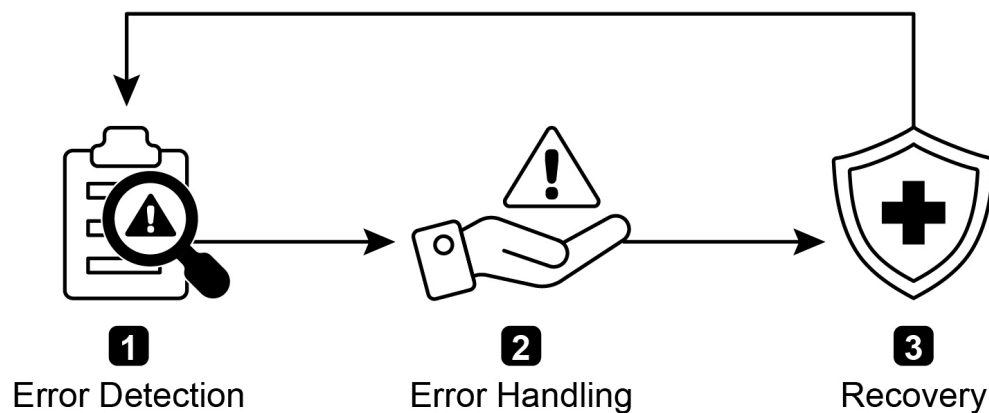


Fig.1: Key components of exception handling and recovery for AI agents

Error Detection: This involves meticulously identifying operational issues as they arise. This could manifest as invalid or malformed tool outputs, specific API errors such as 404 (Not Found) or 500 (Internal Server Error) codes, unusually long response times from services or APIs, or incoherent and nonsensical responses that deviate from expected formats. Additionally, monitoring by other agents or specialized monitoring systems might be implemented for more proactive anomaly detection, enabling the system to catch potential issues before they escalate.

Error Handling: Once an error is detected, a carefully thought-out response plan is essential. This includes recording error details meticulously in logs for later debugging and analysis (logging). Retrying the action or request, sometimes with slightly adjusted parameters, may be a viable strategy, especially for transient errors (retries). Utilizing alternative strategies or methods (fallbacks) can ensure that some functionality is maintained. Where complete recovery is not immediately possible, the agent can maintain partial functionality to provide at least some value (graceful

degradation). Finally, alerting human operators or other agents might be crucial for situations that require human intervention or collaboration (notification).

Recovery: This stage is about restoring the agent or system to a stable and operational state after an error. It could involve reversing recent changes or transactions to undo the effects of the error (state rollback). A thorough investigation into the cause of the error is vital for preventing recurrence. Adjusting the agent's plan, logic, or parameters through a self-correction mechanism or replanning process may be needed to avoid the same error in the future. In complex or severe cases, delegating the issue to a human operator or a higher-level system (escalation) might be the best course of action.

Implementation of this robust exception handling and recovery pattern can transform AI agents from fragile and unreliable systems into robust, dependable components capable of operating effectively and resiliently in challenging and highly unpredictable environments. This ensures that the agents maintain functionality, minimize downtime, and provide a seamless and reliable experience even when faced with unexpected issues.

Practical Applications & Use Cases

Exception Handling and Recovery is critical for any agent deployed in a real-world scenario where perfect conditions cannot be guaranteed.

- **Customer Service Chatbots:** If a chatbot tries to access a customer database and the database is temporarily down, it shouldn't crash. Instead, it should detect the API error, inform the user about the temporary issue, perhaps suggest trying again later, or escalate the query to a human agent.
- **Automated Financial Trading:** A trading bot attempting to execute a trade might encounter an "insufficient funds" error or a "market closed" error. It needs to handle these exceptions by logging the error, not repeatedly trying the same invalid trade, and potentially notifying the user or adjusting its strategy.
- **Smart Home Automation:** An agent controlling smart lights might fail to turn on a light due to a network issue or a device malfunction. It should detect this failure, perhaps retry, and if still unsuccessful, notify the user that the light could not be turned on and suggest manual intervention.
- **Data Processing Agents:** An agent tasked with processing a batch of documents might encounter a corrupted file. It should skip the corrupted file, log the error, continue processing other files, and report the skipped files at the end rather than halting the entire process.

- **Web Scraping Agents:** When a web scraping agent encounters a CAPTCHA, a changed website structure, or a server error (e.g., 404 Not Found, 503 Service Unavailable), it needs to handle these gracefully. This could involve pausing, using a proxy, or reporting the specific URL that failed.
- **Robotics and Manufacturing:** A robotic arm performing an assembly task might fail to pick up a component due to misalignment. It needs to detect this failure (e.g., via sensor feedback), attempt to readjust, retry the pickup, and if persistent, alert a human operator or switch to a different component.

In short, this pattern is fundamental for building agents that are not only intelligent but also reliable, resilient, and user-friendly in the face of real-world complexities.

Hands-On Code Example (ADK)

Exception handling and recovery are vital for system robustness and reliability. Consider, for instance, an agent's response to a failed tool call. Such failures can stem from incorrect tool input or issues with an external service that the tool depends on.

```
from google.adk.agents import Agent, SequentialAgent

# Agent 1: Tries the primary tool. Its focus is narrow and clear.
primary_handler = Agent(
    name="primary_handler",
    model="gemini-2.0-flash-exp",
    instruction="""
Your job is to get precise location information.
Use the get_precise_location_info tool with the user's provided
address.
""",
    tools=[get_precise_location_info]
)

# Agent 2: Acts as the fallback handler, checking state to decide its
action.
fallback_handler = Agent(
    name="fallback_handler",
    model="gemini-2.0-flash-exp",
    instruction="""
Check if the primary location lookup failed by looking at
state["primary_location_failed"].
- If it is True, extract the city from the user's original query and
use the get_general_area_info tool.
- If it is False, do nothing.
""",
```

```

        tools=[get_general_area_info]
    )

# Agent 3: Presents the final result from the state.
response_agent = Agent(
    name="response_agent",
    model="gemini-2.0-flash-exp",
    instruction="""
Review the location information stored in state["location_result"].
Present this information clearly and concisely to the user.
If state["location_result"] does not exist or is empty, apologize
that you could not retrieve the location.
""",
    tools=[] # This agent only reasons over the final state.
)

# The SequentialAgent ensures the handlers run in a guaranteed order.
robust_location_agent = SequentialAgent(
    name="robust_location_agent",
    sub_agents=[primary_handler, fallback_handler, response_agent]
)

```

This code defines a robust location retrieval system using a ADK's SequentialAgent with three sub-agents. The primary_handler is the first agent, attempting to get precise location information using the get_precise_location_info tool. The fallback_handler acts as a backup, checking if the primary lookup failed by inspecting a state variable. If the primary lookup failed, the fallback agent extracts the city from the user's query and uses the get_general_area_info tool. The response_agent is the final agent in the sequence. It reviews the location information stored in the state. This agent is designed to present the final result to the user. If no location information was found, it apologizes. The SequentialAgent ensures that these three agents execute in a predefined order. This structure allows for a layered approach to location information retrieval.

At a Glance

What: AI agents operating in real-world environments inevitably encounter unforeseen situations, errors, and system malfunctions. These disruptions can range from tool failures and network issues to invalid data, threatening the agent's ability to complete its tasks. Without a structured way to manage these problems, agents can be fragile, unreliable, and prone to complete failure when faced with unexpected